

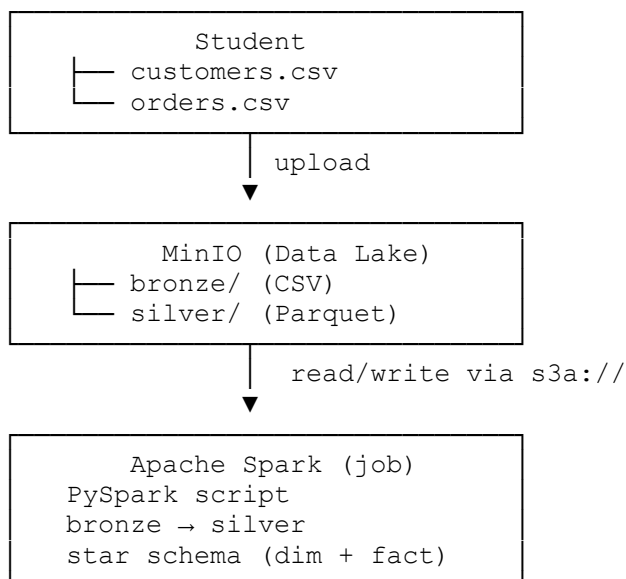
Lab: Spark + MinIO — Processing Data from Bronze to Silver (Star Schema)

Goal of the exercise

In this lab you will:

- run an **Apache Spark + MinIO** environment in Docker containers,
 - **manually upload raw data (CSV)** into the **bronze** layer in MinIO,
 - prepare a **Spark job** that:
 - reads data from the bronze layer,
 - transforms it and builds a **star schema** (dimensions + facts),
 - writes the results to the **silver** layer as **Parquet** files.
-

Solution architecture



Steps to complete

◆ Step 1. Create the project

Create a project directory, e.g.:

```
mkdir spark-minio-star
cd spark-minio-star
```

Inside it, create the folder structure:

```
spark-minio-star/  
├─ docker-compose.yml  
├─ .env  
├─ spark/  
│   ├── Dockerfile  
│   ├── requirements.txt  
│   └─ app/  
│       └─ job_silver_star.py  
└─ data/  
    └─ bronze/  
        ├── customers.csv  
        └─ orders.csv
```

◆ Step 2. Prepare input data (bronze)

Create two CSV files (in data/bronze):

customers.csv

```
customer_id,first_name,last_name,email,country  
1,Ada,Lovelace,ada@example.com,UK  
2,Alan,Turing,alan@example.com,UK  
3,Grace,Hopper,grace@example.com,US  
4,Marie,Curie,marie@example.com,FR
```

orders.csv

```
order_id,customer_id,order_timestamp,order_amount,currency  
1001,1,2025-10-01 10:15:00,120.50,GBP  
1002,2,2025-10-02 12:00:00,200.00,GBP  
1003,3,2025-10-03 09:30:00,99.99,USD  
1004,1,2025-10-05 16:45:00,50.00,GBP  
1005,4,2025-10-06 08:10:00,300.00,EUR
```

◆ Step 3. Create the .env file

This file centralizes configuration:

```
# MinIO  
MINIO_ROOT_USER=admin  
MINIO_ROOT_PASSWORD=adminadmin  
MINIO_ENDPOINT=http://minio:9000  
MINIO_BROWSER_PORT=9001  
  
# S3 bucket + prefixes  
S3_BUCKET=lake  
S3_BRONZE_PREFIX=bronze  
S3_SILVER_PREFIX=silver  
  
# Spark  
SPARK_APP_NAME=bronze_to_silver_star  
SPARK_MASTER=local[*]
```

◆ Step 4. Prepare `docker-compose.yml`

This file runs **MinIO** and the **Spark job** in two containers:

```
version: "3.8"

services:
  minio:
    image: minio/minio:RELEASE.2024-12-07T00-00-00Z
    container_name: minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio_data:/data

  spark-job:
    build:
      context: ./spark
      dockerfile: Dockerfile
    container_name: spark-job
    depends_on:
      - minio
    environment:
      SPARK_MASTER: ${SPARK_MASTER}
      SPARK_APP_NAME: ${SPARK_APP_NAME}
      AWS_ACCESS_KEY_ID: ${MINIO_ROOT_USER}
      AWS_SECRET_ACCESS_KEY: ${MINIO_ROOT_PASSWORD}
      S3_ENDPOINT: ${MINIO_ENDPOINT}
      S3_BUCKET: ${S3_BUCKET}
      S3_BRONZE_PREFIX: ${S3_BRONZE_PREFIX}
      S3_SILVER_PREFIX: ${S3_SILVER_PREFIX}
    command: >
      bash -lc "
        for i in {1..60}; do
          nc -z minio 9000 && break || { echo 'Waiting for MinIO...'; sleep
2; }
        done;
        /opt/spark/bin/spark-submit
          --master ${SPARK_MASTER}
          --name ${SPARK_APP_NAME}
          --conf spark.hadoop.fs.s3a.endpoint=${S3_ENDPOINT}
          --conf spark.hadoop.fs.s3a.access.key=${AWS_ACCESS_KEY_ID}
          --conf spark.hadoop.fs.s3a.secret.key=${AWS_SECRET_ACCESS_KEY}
          --conf spark.hadoop.fs.s3a.path.style.access=true
          --conf spark.hadoop.fs.s3a.connection.ssl.enabled=false
          /app/job_silver_star.py
      "

volumes:
  minio_data:
```

◆ Step 5. Spark image — `spark/Dockerfile`

```
FROM eclipse-temurin:11-jre-jammy

ARG SPARK_VERSION=3.5.1
ARG HADOOP_PROFILE=hadoop3
ARG HADOOP_AWS_VERSION=3.3.6
ARG AWS_SDK_BUNDLE_VERSION=1.12.262

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 python3-pip curl ca-certificates tini bash netcat-openbsd \
    && rm -rf /var/lib/apt/lists/*

ENV SPARK_HOME=/opt/spark
ENV PATH=$SPARK_HOME/bin:$PATH

RUN curl -fsSL https://archive.apache.org/dist/spark/spark-
${SPARK_VERSION}/spark-${SPARK_VERSION}-bin-${HADOOP_PROFILE}.tgz \
    -o /tmp/spark.tgz \
    && mkdir -p ${SPARK_HOME} \
    && tar -xzf /tmp/spark.tgz -C /opt \
    && mv /opt/spark-${SPARK_VERSION}-bin-${HADOOP_PROFILE} ${SPARK_HOME} \
    && rm -f /tmp/spark.tgz

# s3a libraries
RUN mkdir -p ${SPARK_HOME}/jars \
    && curl -fsSL https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-
aws/${HADOOP_AWS_VERSION}/hadoop-aws-${HADOOP_AWS_VERSION}.jar \
    -o ${SPARK_HOME}/jars/hadoop-aws-${HADOOP_AWS_VERSION}.jar \
    && curl -fsSL https://repo1.maven.org/maven2/com/amazonaws/aws-java-sdk-
bundle/${AWS_SDK_BUNDLE_VERSION}/aws-java-sdk-bundle-
${AWS_SDK_BUNDLE_VERSION}.jar \
    -o ${SPARK_HOME}/jars/aws-java-sdk-bundle-
${AWS_SDK_BUNDLE_VERSION}.jar

WORKDIR /opt/app
COPY requirements.txt /opt/app/requirements.txt
RUN pip3 install --no-cache-dir -r requirements.txt

COPY app /app

ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["bash"]
```

`spark/requirements.txt`

```
pyspark==3.5.1
pyarrow>=14,<19
```

◆ Step 6. Spark script — `spark/app/job_silver_star.py`

This script transforms bronze → silver using a star schema:

```
import os
from pyspark.sql import SparkSession, functions as F

S3_ENDPOINT = os.getenv("S3_ENDPOINT")
```

```

AWS_ACCESS_KEY_ID = os.getenv("AWS_ACCESS_KEY_ID")
AWS_SECRET_ACCESS_KEY = os.getenv("AWS_SECRET_ACCESS_KEY")
BUCKET = os.getenv("S3_BUCKET", "lake")
BRONZE = os.getenv("S3_BRONZE_PREFIX", "bronze")
SILVER = os.getenv("S3_SILVER_PREFIX", "silver")

def s3url(path): return f"s3a://{BUCKET}/{path}"

def build_spark():
    spark = SparkSession.builder.appName("BronzeToSilver").getOrCreate()
    conf = spark._jsc.hadoopConfiguration()
    conf.set("fs.s3a.endpoint", S3_ENDPOINT)
    conf.set("fs.s3a.access.key", AWS_ACCESS_KEY_ID)
    conf.set("fs.s3a.secret.key", AWS_SECRET_ACCESS_KEY)
    conf.set("fs.s3a.path.style.access", "true")
    conf.set("fs.s3a.connection.ssl.enabled", "false")
    return spark

def main():
    spark = build_spark()

    # --- Read bronze data ---
    customers = spark.read.option("header",
True).csv(s3url(f"{BRONZE}/customers.csv"))
    orders = spark.read.option("header",
True).csv(s3url(f"{BRONZE}/orders.csv"))

    # --- Dim Customer ---
    dim_customer = (
        customers
        .withColumn("customer_id", F.col("customer_id").cast("long"))
        .withColumn("customer_name", F.concat_ws(" ", "first_name",
"last_name"))
        .select(
            F.col("customer_id").alias("customer_key"),
            "customer_name",
            F.lower("email").alias("email"),
            F.upper("country").alias("country")
        )
        .dropDuplicates(["customer_key"])
    )

    # --- Dim Date ---
    orders = orders.withColumn("order_timestamp",
F.to_timestamp("order_timestamp"))
    dim_date = (
        orders.select(F.to_date("order_timestamp").alias("order_date"))
        .dropna().dropDuplicates()
        .withColumn("date_key",
F.date_format("order_date", "yyyyMMdd").cast("int"))
        .withColumn("year", F.year("order_date"))
        .withColumn("month", F.month("order_date"))
    )

    # --- Fact Orders ---
    fact_orders = (
        orders
        .join(dim_customer, F.col("customer_id")==F.col("customer_key"),
"left")
        .join(dim_date, F.to_date("order_timestamp")==F.col("order_date"),
"left")
    )

```

```

        .select(
            F.col("order_id").cast("long"),
            F.col("customer_key"),
            F.col("date_key"),
            F.col("order_amount").cast("double").alias("amount"),
            F.col("currency").alias("currency"),
            F.col("order_timestamp")
        )
    )

    # --- Write silver ---

dim_customer.write.mode("overwrite").parquet(s3url(f"{SILVER}/star/dim_customer"))

dim_date.write.mode("overwrite").parquet(s3url(f"{SILVER}/star/dim_date"))

fact_orders.write.mode("overwrite").partitionBy("currency").parquet(s3url(f"{SILVER}/star/fact_orders"))

    print("✅ Silver layer written to MinIO.")
    spark.stop()

if __name__ == "__main__":
    main()

```

◆ Step 7. Run the environment

1. Start:
 2. `docker compose up --build`
 3. Open **MinIO Console**: <http://localhost:9001>
Login: admin, password: adminadmin
 4. In MinIO Console:
 - create the **bucket** lake,
 - create a folder bronze/ inside it,
 - **upload** customers.csv and orders.csv (from Step 2).
 5. After uploading the files, restart the job:
 6. `docker compose restart spark-job`
-

◆ Step 8. Validate the results

You should see new folders in MinIO (bucket lake):

```

silver/star/
├─ dim_customer/
├─ dim_date/
└─ fact_orders/

```

Each contains Parquet (.parquet) files.

You can inspect them via **MinIO Console**, or download and open them with **Apache Parquet Viewer** or **Python/pandas**.